

Compiler Design Laboratory

Assignment: Syntax Directed Translation using Flex and Bison

1. Objective

The objective of this assignment is to:

- Implement a parser using **Flex and Bison**
- Apply **Syntax Directed Translation (SDT)**
- Generate:
 - Postfix notation
 - Syntax Tree
 - Three Address Code (TAC)

2. Problem Statement

Design and implement a compiler front-end for arithmetic expressions using Flex and Bison. The system should:

- Parse arithmetic expressions
- Generate postfix notation
- Construct syntax tree
- Generate three-address intermediate code

3. Grammar

Use the following grammar:

```
E -> E + T
E -> E - T
E -> T

T -> T * F
T -> T / F
T -> F

F -> (E)
F -> id
```

4. Syntax Directed Translation (SDT)

Attach semantic actions to generate:

- Postfix notation
- Temporary variables for TAC

Example:

```
E -> E1 + T    { print(E1.code || T.code || '+' ) }
```

5. Flex Specification (lexer.l)

```
%{
#include "parser.tab.h"
%}

%%

/* When a number is matched:
   yytext contains the matched string (e.g., "123")
   atoi(yytext) converts it to integer 123
   yylval stores this value so that Bison can use it
*/
[0-9]+      { yylval = atoi(yytext); return NUMBER; }

/* Match identifiers (variable names like a, x, etc.) */
[a-zA-Z]+   { return ID; }

/* Return operators directly as tokens */
"+"         { return '+'; }
"-"         { return '-'; }
"*"         { return '*'; }
"/"         { return '/'; }
"("         { return '('; }
")"         { return ')'; }

/* Ignore whitespace (spaces, tabs, newlines) */
[ \t\n]     ;

/* Default rule:
   Matches any single character not handled above.
   yytext[0] gives that character.
   This ensures the lexer does not get stuck and
   passes unknown symbols to the parser.
*/
.           { return yytext[0]; }

%%

/* Indicates end of input to Flex */
int yywrap() { return 1; }
```

6. Bison Specification (parser.y)

```
%{
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

/* Global counter used to generate unique temporary variables */
int tempCount = 0;

/* Function to generate temporary variables (t0, t1, t2, ...) */
char* newTemp() {
    char *t = malloc(10);
    sprintf(t, "t%d", tempCount++);
    return t;
}
%}

%token NUMBER ID

%%

E : E '+' T { printf("+_"); }
  | E '-' T { printf("-_"); }
  | T
  ;

T : T '*' F { printf("*_"); }
  | T '/' F { printf("/_"); }
  | F
  ;

F : '(' E ')'
  | NUMBER { printf("%d_", $1); }
  | ID      { printf("id_"); }
  ;

%%

int main() {
    printf("Enter expression:\n");
    yyparse();
    return 0;
}

/* Error handling function called by Bison
- Automatically invoked when a syntax error occurs
- 's' contains the error message (usually "syntax error")
- Used to report invalid input to the user
```

```

*/
int yyerror(char *s) {
    printf("Syntax Error\n");
    return 0;
}

```

7. Tasks to Perform

Students must implement the following:

1. Write `lexer.l` using Flex.
2. Write `parser.y` using Bison with SDT rules.
3. Modify SDT rules to:
 - Generate postfix notation
 - Generate three-address code
 - Build syntax tree (node-based)
4. Implement temporary variable generation.
5. Print syntax tree (in preorder or inorder).

8. Compilation Steps

```

bison -d parser.y
flex lexer.l
gcc parser.tab.c lex.yy.c -o parser -lfl
./parser      /* After running, type the input expression, press
               Enter,
               then press Ctrl+D (EOF) to start parsing */

```

9. Sample Inputs

Valid Inputs

```

a+b*c
(3+4)*5
x*y+z

```

Invalid Inputs

```

a+*b
(3+4
*5+

```

10. Expected Output

Postfix Example

```
Input: a+b*c
Output: a b c * +
```

Three Address Code Example

```
t1 = b * c
t2 = a + t1
```

Hint: Building Syntax Tree (Node-Based)

Students are required to construct a syntax tree using a node-based structure. The following hints will help guide the implementation.

1. Define a Node Structure

Each node in the syntax tree should represent either:

- an operator (+, -, *, /)
- an operand (number or identifier)

```
typedef struct Node {
    char value[10];
    struct Node *left;
    struct Node *right;
} Node;
```

2. Create Node Functions

You may define helper functions to create nodes:

```
Node* createLeaf(char *val);
Node* createNode(char *op, Node *left, Node *right);
```

3. Modify Bison Grammar with Semantic Values

Use %union to store node pointers:

```
%union {
    int num;
    char *id;
    struct Node *node;
}
```

Assign types to tokens and non-terminals:

```
%type <node> E T F
```

Explanation of %type <node> E T F

- This declaration tells Bison that the non-terminals E, T, and F will carry semantic values of type `node`.
- The type `node` is defined inside the %union as a pointer to a syntax tree node (e.g., `struct Node*`).
- During parsing, each of these non-terminals will return a pointer to a syntax tree node.

- This allows us to build the syntax tree by combining child nodes using semantic actions.

Example Usage

```
E : E '+' T {
    $$ = createNode("+", $1, $3);
}
```

- \$1 and \$3 are nodes returned by E and T.
- \$\$ stores the resulting node for the current production.
- The declaration %type <node> ensures type consistency.

4. Attach Semantic Actions

Modify grammar rules to build tree nodes:

```
E : E '+' T {
    $$ = createNode("+", $1, $3);
}
| T {
    $$ = $1;
}
;
```

Similarly implement for other productions.

5. Leaf Node Creation

For numbers and identifiers:

```
F : NUMBER {
    $$ = createLeaf("num");
}
| ID {
    $$ = createLeaf("id");
}
;
```

Explanation of \$1, \$2, \$3, \$\$ in Bison

In Bison, each grammar rule can have associated semantic values. These are accessed using special symbols:

- \$1, \$2, \$3, ... represent the semantic values of symbols on the **right-hand side (RHS)** of a production.
- \$\$ represents the semantic value of the **left-hand side (LHS)** of the production.

General Form

```
A : B C D {
    $$ = some_function($1, $2, $3);
}
```

- \$1 → value of B
- \$2 → value of C
- \$3 → value of D
- \$\$ → value of A

Example: Expression Parsing

```
E : E '+' T {
    $$ = createNode("+", $1, $3);
}
```

- \$1 → syntax tree of left expression E
- \$3 → syntax tree of T
- \$\$ → new syntax tree node combining \$1 and \$3

Important Notes

- The numbering (\$1, \$2, ...) follows the order of symbols in the RHS.
- The type of \$1, \$2, etc., depends on declarations in %union and %type.
- \$\$ must be assigned a value for the production to return a meaningful result.

6. Store Root of the Tree

The final tree root will be available as:

```
E (start symbol)
```

You can store it in a global pointer if needed.

7. Tree Traversal (Printing)

Implement one of the following traversals:

- Preorder (Prefix form)
- Inorder (Expression form)
- Postorder (Postfix form)

Example:

```
void inorder(Node *root);
void preorder(Node *root);
void postorder(Node *root);
```

8. Expected Output (Example)

For input:

```
a + b * c
```

Possible tree:

```

      +
     / \
    a  *
       / \
      b  c
```

9. Important Notes

- Allocate memory using malloc()
- Ensure proper pointer connections
- Each grammar rule should return a node
- Avoid memory leaks (optional for this lab)

10. Goal

At the end, your parser should:

- Build a syntax tree
- Traverse and print it
- Use it for further processing (TAC generation)

11. Advanced Task (For Next Lab)

- Implement expression evaluation
- Add operator precedence using %left
- Extend grammar for unary operators

12. Submission Requirements

- lexer.l file
- parser.y file
- Output screenshots
- Explanation of SDT rules

13. Learning Outcomes

After completing this assignment, students will be able to:

- Apply syntax-directed translation
- Generate intermediate representations
- Build compiler front-end components
- Integrate Flex and Bison effectively

— End of Assignment —